

Practical - Week - 11

Task-1

Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0; // Shared variable

void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Multithreaded Counter - Race Condition\n");
    printf("-----\n");

    /* Create threads */
    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    /* Wait for all threads */
    for (int i = 0; i < NUM_THREADS; i++)
    {
        /* Wait for all threads */
        for (int i = 0; i < NUM_THREADS; i++)
        {
            if (pthread_join(threads[i], NULL) != 0)
            {
                perror("pthread_join");
                return 1;
            }
        }

        printf("Expected counter value : %d\n",
              NUM_THREADS * INCREMENTS);

        printf("Actual counter value   : %d\n", counter);

        if (counter != NUM_THREADS * INCREMENTS)
        {
            printf("Race condition detected!\n");
        }
        else
        {
            printf("Counter reached expected value.\n");
        }

        return 0;
    }
}

root@Priylnv:~# mkdir server >
root@Priylnv:~# gcc server.c -o server -pthread
root@Priylnv:~# ./server
Multiclient server started on port 8080...
-----,-----
Client connected: 127.0.0.1:4042
Data received: Success!

```

Observation:

Multiple threads were created using `pthread_create()` and executed concurrently to update a shared counter. Since the counter was accessed without synchronization, a race condition occurred, resulting in an actual counter value that may differ from the expected value.

Task-2

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
GNU nano 7.2 cc
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;

/* Create mutex */
pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        /* Lock before accessing shared variable */
        pthread_mutex_lock(&lock);

        counter++;

        /* Unlock after updating */
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    /* Initialize mutex */
    pthread_mutex_init(&lock, NULL);

    printf("Multithreaded Counter - Mutex Synchronization\n");
    printf("-----\n");

    /* Create threads */
    for (int i = 0; i < NUM_THREADS; i++)
```

```

root@Priylnv:~# nano counter_mutex.c
root@Priylnv:~# gcc counter_mutex.c -o counter_mutex -pthread
root@Priylnv:~# ./counter_mutex
Multithreaded Counter - Mutex Synchronization
-----
Expected counter value : 500000
Actual counter value   : 500000
Counter value is correct.
Race condition prevented using mutex.
root@Priylnv:~# |

```

```

/* Create threads */
for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_create(&threads[i], NULL,
                      increment_counter, NULL) != 0)
    {
        perror("pthread_create");
        return 1;
    }
}

/* Wait for all threads */
for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_join(threads[i], NULL) != 0)
    {
        perror("pthread_join");
        return 1;
    }
}

printf("Expected counter value : %d\n",
       NUM_THREADS * INCREMENTS);

printf("Actual counter value   : %d\n", counter);

if (counter == NUM_THREADS * INCREMENTS)
{
    printf("Counter value is correct.\n");
    printf("Race condition prevented using mutex.\n");
}

/* Destroy mutex */
pthread_mutex_destroy(&lock);

return 0;
}

```

Observation:

Mutex locks were used to synchronize access to the shared counter. `pthread_mutex_lock()` allowed only one thread to update the counter at a time, while `pthread_mutex_unlock()` released the lock. The synchronized program produced the expected counter value and prevented the race condition, although synchronization introduces some execution overhead.